

School of Computer Science Engineering and Information Systems
Department of Information Technology

Fall Semester 2026-27
CLOUD COMPUTING BITE412L
PROJECT REPORT

**CLOUDRELIEF: A SMART DISASTER RESPONSE AND RELIEF
MANAGEMENT SYSTEM WITH COMPARATIVE SERVERLESS
AND
TRADITIONAL CLOUD ARCHITECTURE**

Submitted by

Sanjay Chidambaram	23BIT0425
Adithya Prabakaran	23BIT0102
M R Yajunesh	23BIT0399

Under the Guidance of
Siva Rama Krishnan S
Associate Professor Grade 1
VIT, Vellore.

Fall Semester 2026-27

TABLE OF CONTENTS

S.No.	Contents	Page No.
1	Abstract	
2	Chapter 1 : Introduction	4
3	Chapter 2 : Literature Survey	6
4	Chapter 3 : Proposed Technique	9
5	Chapter 4 : Results and Discussion	12
6	Chapter 5 : Conclusion & Future Work	13
7	References	14

ABSTRACT

In a flood, a fire, or a building collapse, the first hour is usually the one that decides how things turn out. Right now, most disaster reporting still runs on phone calls and word of mouth, and there is no automatic way to tell a genuine emergency apart from a routine complaint until someone has already spent time reading through it. CloudRelief is our attempt at fixing that: a cloud-native reporting system where a citizen can log an incident with a location, a short description and a photo, and the system itself works out roughly how severe it is — using AI-based image classification — before anyone in the response team even opens the dashboard.

What makes this more than an application build is that we deliberately built the same reporting workflow twice. One version runs entirely serverless on AWS — API Gateway, Lambda, DynamoDB, S3, Cognito, SNS and Rekognition working together. The other is the old-fashioned way: a single always-on virtual machine running a Flask/Django application with its own database, no auto-scaling, nothing fancy. We then throw identical, increasingly heavy simulated traffic at both using Locust and measure what actually happens to latency, error rate and cost as the load climbs from 10 to 500 concurrent users.

The point of doing it this way is that it turns a set of cloud computing concepts we were taught — service models, virtualization, storage, security, elasticity — into something we can actually show working, side by side, with numbers behind it rather than a slide that just asserts serverless is better. Once the benchmark run is complete, the outcome should be a working disaster-response platform and a small, reusable way of testing serverless against traditional deployment for this kind of bursty, safety-critical traffic, which is useful well past this one project.

CHAPTER 1: INTRODUCTION

Reporting a disaster used to be limited mainly by whether a phone line was working. The harder problem now is what happens after the report comes in — turning a pile of scattered, unstructured messages into something a response team can actually act on before it is too late. That is the gap this project tries to close, and we chose to close it in a way that also lets us test something about cloud architecture itself, not just about disaster reporting.

1.1 Motivation

Two things pushed us toward this particular project.

- The reporting gap. During a flood, a fire or a structural emergency, citizens have no single fast channel to report what they are seeing. Response teams end up piecing information together from calls and radio chatter, with no automated way to know which report needs attention first.
- The architectural question we kept running into in class. Cloud computing material — ours included — tends to state that serverless is cheaper and scales better than a traditional deployment, but that claim is usually presented as settled rather than measured against a specific workload. We wanted to actually check it instead of repeating it.

1.2 Scope

- What we built: a cloud-native disaster reporting platform with a citizen portal, an admin portal, and AI-based severity triage, deployed identically on two different compute architectures.
- The AI part: we use Amazon Rekognition's pretrained image classification rather than training our own model, mainly because there is no public dataset of citizen-submitted disaster photographs large enough to train a classifier from scratch.
- The comparison: both deployments run the same citizen-reporting workflow. The only thing that changes between them is the compute model, which is what lets us isolate that one variable.

1.3 Problem Statement

Citizens facing a flood, a fire or a structural emergency have no fast, unified way to report it, and response teams have no automated way to judge severity across the reports coming in. Separately, teams that pick a cloud deployment model — serverless or traditional — usually do so on the strength of general industry claims rather than evidence measured against their own workload. This project sets out to build a working, AI-assisted reporting system and, alongside it, a fair comparison of the two deployment models available to build it.

1.4 Objectives

1. Build a cloud-native disaster reporting platform with automatic AI-based severity triage.
2. Deploy the identical citizen-reporting workflow on a serverless (FaaS) architecture and a traditional virtual-machine (IaaS) architecture.
3. Benchmark both deployments under identical simulated load using a controlled load-testing harness.
4. Compare the cost, latency and scalability of both deployment models using the measured results, not assumptions.

5. Show a practical, working example of core cloud computing concepts: service models, virtualization, storage, security and elasticity.

1.5 Organization of the Report

- Chapter 1: Introduction – the motivation, problem statement and objectives.
- Chapter 2: Literature Survey – existing work on serverless disaster systems, FaaS versus IaaS performance, and AI-based disaster image classification.
- Chapter 3: Proposed Technique – the architecture for both tracks, module design, algorithms, dataset and tools.
- Chapter 4: Results and Discussion – experimental setup, evaluation methodology and performance metrics.
- Chapter 5: Conclusion – summary and future work.

CHAPTER 2: LITERATURE SURVEY

2.1 Overview

Cloud-based disaster management has moved, over roughly the last decade, from being treated as a plain IT storage-and-processing problem toward event-driven, serverless designs — largely because a real disaster produces demand that is unpredictable and bursty, and nobody wants infrastructure sitting provisioned and idle the rest of the time waiting for it. At the same time, deep learning has made it realistic to classify a disaster photo automatically instead of needing a human to look at it first. Reading through the literature, though, most work picks one side of this or the other: either the cloud-architecture question, or the AI-classification question. A controlled comparison of serverless against a traditional deployment, for one specific application, is harder to find than we expected.

2.2 Review of Existing Methods

We looked at seven papers that, between them, cover serverless disaster systems, FaaS/IaaS performance comparison, and AI-based disaster image classification.

Feasibility of Serverless Cloud Services for Disaster Management Information Systems (Asghar et al., 2018)

Asghar and colleagues look at whether serverless cloud services make sense specifically for disaster management information systems, weighing up the cost and operational trade-offs. Their conclusion leans strongly toward serverless being the more sensible choice for this kind of workload on cost grounds alone — but the paper stops at the argument; there is no implemented system and no measured comparison against a traditional deployment to back it up.

TEMA: Event Driven Serverless Workflows Platform for Natural Disaster Management (Sicari et al., 2023)

TEMA is a purpose-built serverless workflow platform for natural disaster management. It deals directly with a practical annoyance of serverless design: functions cannot call each other directly, so data has to be passed through external storage between steps. What we found most useful here is the empirical measurement — cold starts and this inter-function data passing turn out to account for a meaningful chunk of end-to-end latency, which is exactly the kind of overhead our own Track A is likely to hit.

Serverless Computing for Cloud-Based Power Grid Emergency Generation Dispatch (Zhang, Luo & Litvinov, 2020)

This one is further from our domain but still relevant: an event-driven, serverless architecture for emergency generation dispatch on a power grid, actually deployed and tested at a real grid operator (ISO New England). It shows that a FaaS design can give fault-tolerant emergency response independent of on-premises infrastructure at a low running cost — useful evidence that the approach works in a live, safety-critical setting, even though the domain (power grids) is not ours and the paper does not benchmark against a traditional deployment either.

Towards Demystifying Serverless Machine Learning Training (Jiang et al., 2021)

Out of everything we read, this is the most rigorous FaaS-versus-IaaS comparison we found. Jiang and colleagues built LambdaML specifically to keep the comparison fair between the two deployment models for distributed ML training, and their headline finding is one we borrow directly for our own analysis framework in Chapter 4: FaaS can be faster than IaaS, but when it is faster it is rarely also cheaper, because the pay-per-invocation pricing eats into the advantage under sustained load. The catch for us is that their methodology was built for ML training workloads, and we cannot assume it transfers cleanly to bursty, low-compute-per-request reporting traffic — which is part of why we are running our own benchmark rather than relying on theirs.

Deep-Learning-Based Aerial Image Classification for Emergency Response Applications Using UAVs (Kyrkou & Theocharides, 2019)

This paper is the closest thing to a precedent for our AI component. Kyrkou and Theocharides built the AIDER dataset and a lightweight CNN called ERNet that classifies disaster images into fire/smoke, flood, collapsed building/rubble, traffic accident and normal, hitting close to 90% accuracy while running efficiently on embedded hardware. The categories line up almost exactly with what we classify in CloudRelief. The difference is that their target is on-board UAV inference, not a cloud-hosted pipeline reacting to citizen-submitted photos, so we could not reuse their model directly.

Creating xBD: A Dataset for Assessing Building Damage from Satellite Imagery (Gupta et al., 2019)

xBD is a large, well-known satellite-imagery dataset with graded building-damage labels, and it is the standard benchmark most building-damage classification work compares itself against. It was not directly usable for us because it is built around pre- and post-event satellite image pairs, and our input is a single ground-level photo taken by a citizen on a phone — a very different image format.

Emergency Management Systems Using Mobile Cloud Computing: A Survey (Nanda et al., 2023)

This survey covers the overlap between mobile cloud computing and emergency management systems and lays out a proposed architecture, flagging resource-constrained devices, fast data transfer and efficient resource use as the main design challenges. It is a useful map of the problem space, but it stays conceptual — there is no implementation and nothing benchmarked.

2.3 Comparative Analysis

Paper Title & Year	Technique Used	Domain / Dataset	Advantages	Limitations
Feasibility of Serverless Cloud Services for Disaster Management Information Systems (2018)	Economic and feasibility analysis of serverless adoption	Disaster management information systems	Makes a strong cost case for serverless in this domain	No implemented system or benchmark against a traditional deployment
TEMA: Event Driven Serverless Workflows Platform for Natural Disaster Management (2023)	Event-driven serverless workflow orchestration	Natural disaster management workflows	Purpose-built serverless workflow platform with measured cold-start data	Cold starts and inter-function data passing add real latency overhead
Serverless Computing for Cloud-Based Power Grid Emergency Generation Dispatch (2020)	Event-driven FaaS architecture, deployed at a real grid operator	Power grid emergency dispatch	Proven, low-cost emergency response independent of on-premise infrastructure	Specific to power grids; not citizen-facing and not benchmarked against IaaS
Towards Demystifying Serverless Machine Learning Training (2021)	Controlled FaaS vs. IaaS benchmark platform (LambdaML)	Distributed ML training workloads	The most rigorous published FaaS/IaaS methodology we found	Built for ML training, not for bursty, safety-critical reporting traffic
Deep-Learning-Based Aerial Image Classification for Emergency Response Applications Using UAVs (2019)	Lightweight CNN (ERNet) trained on the custom AIDER dataset	Aerial disaster imagery: fire, flood, collapsed building	About 90% accuracy at real-time speed on embedded hardware	Built for on-board UAV inference, not a cloud-hosted reporting pipeline

Creating xBD: A Dataset for Assessing Building Damage from Satellite Imagery (2019)	Large-scale labelled satellite-imagery dataset	Post-disaster building damage assessment	Established benchmark dataset with graded damage labels	Built for paired satellite images, not single ground-level citizen photos
Emergency Management Systems Using Mobile Cloud Computing: A Survey (2023)	Literature survey and a proposed conceptual architecture	Mobile cloud computing for EMS	Broad, well-organised survey of the design challenges	Conceptual only — no implemented or benchmarked system

2.4 Research Gap

What is missing: the serverless-for-disaster papers we found (Asghar et al.; Sicari et al.; Zhang, Luo & Litvinov) each build or argue for a single serverless architecture, but none of them put that architecture up against an equivalent traditional deployment of the same workload — the cost and performance advantage of serverless is argued, not measured. The best FaaS-versus-IaaS methodology available (Jiang et al.) exists for a different kind of workload — ML training — not the bursty, high-concurrency, low-per-request-compute pattern of citizen emergency reporting. And on the AI side, the disaster-image classification work (Kyrkou & Theodoridis; Gupta et al.) evaluates the classifier on its own, rather than as one stage in a working, alertable reporting pipeline.

What we are doing about it: CloudRelief tries to close both gaps at once. We build a full citizen-facing reporting pipeline with AI severity triage built in, and — taking a cue from the controlled-comparison discipline in Jiang et al. — we deploy that identical pipeline on both a serverless and a traditional architecture and benchmark both under a load pattern meant to resemble a real disaster-reporting surge. If it works as intended, this gives us both a working system and some actual measured evidence on serverless-versus-traditional trade-offs for this specific kind of workload, which none of the seven papers above quite provide on their own.

CHAPTER 3: PROPOSED TECHNIQUE

3.1 System Architecture

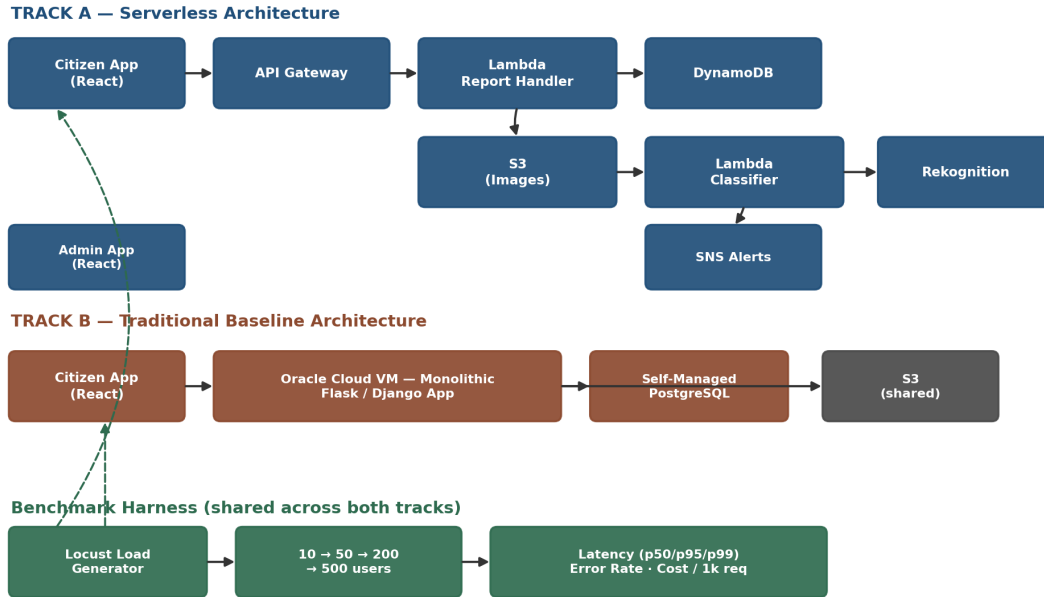


Figure 3.1: Architecture of the CloudRelief Dual-Track System

We built the identical citizen-reporting workflow twice — once serverless, once traditional — plus a benchmarking harness that exercises both at the same time. Figure 3.1 shows the three pieces:

- 1. Track A – Serverless Architecture.** A citizen submits a report from the React frontend, which goes through API Gateway to a Lambda report-handler function. That function writes the incident to DynamoDB, and for image uploads, hands back a pre-signed URL so the photo goes straight to S3 from the citizen's device. The moment the image lands in S3, a second Lambda function fires, calls Rekognition to classify it, and works out a severity score; anything above threshold publishes an alert through SNS. Admins query incidents through the same API Gateway/Lambda pattern, just in reverse.
- 2. Track B – Traditional Baseline Architecture.** The same citizen-facing flow, but built as a single Flask/Django application on one always-on virtual machine (we used Oracle Cloud's Always Free tier), with a self-managed PostgreSQL database on the same box and no auto-scaling at all. It reads and writes to the same S3 bucket Track A uses, so image storage stays constant across both and does not skew the comparison.
- 3. Benchmark Harness.** A Locust load-testing setup fires identical traffic at both tracks in stages — 10, 50, 200 and then 500 simultaneous users — and logs response latency at the 50th, 95th and 99th percentiles, the error rate, and the cost per 1,000 requests at each stage.

3.2 Module-wise Description

3.2.1 Citizen Reporting Module

Handles registration and login through Cognito, incident submission with GPS location and a text description, and image upload through a pre-signed S3 URL. Citizens can also check the status of a report they already submitted.

3.2.2 AI Classification and Severity Scoring Module

Fires automatically once an image lands in S3. Calls Rekognition to classify it, then folds the classifier's confidence together with keyword signals pulled from the citizen's text and how many other reports are coming from nearby, into one severity score that is simple enough to explain if an admin asks why a report was flagged.

3.2.3 Alerting Module

Once a severity score crosses the configured threshold, this module publishes to an SNS topic, which fans the alert out to whoever is subscribed — dashboard, email, SMS — without the function that raised the alert needing to know who is actually listening.

3.2.4 Admin Dashboard Module

Lets authorities see incoming incidents on a map, assign a rescue team, move a report through unassigned → assigned → resolved, and glance at basic stats by severity and type.

3.2.5 Traditional Baseline Module (Track B)

Reimplements the citizen reporting flow as a single-process application with no auto-scaling, purely so its behaviour under load has something to be measured against.

3.2.6 Benchmarking Module

The Locust setup that drives identical request patterns at both tracks with rising concurrency and logs the numbers we analyse in Chapter 4.

3.3 Algorithms Used

For severity scoring we deliberately avoided a second machine-learning model. A weighted-scoring formula is easier to explain when someone asks why a report got flagged, and that mattered more to us here than squeezing out a bit more accuracy from a black box.

Pseudo-code for Severity Scoring:

```
FUNCTION compute_severity(image_labels, description_text, nearby_report_count):
  classifier_score = MAX(confidence FOR label IN image_labels
  IF label IN [Flood, Fire, StructuralDamage])      keyword_score =
COUNT(urgent_keywords IN description_text) * URGENCY_WEIGHT      density_score =
MIN(nearby_report_count / DENSITY_CAP, 1.0) * DENSITY_WEIGHT      severity =
(classifier_score * 0.5) + (keyword_score * 0.25) + (density_score * 0.25)      IF
severity >= ALERT_THRESHOLD:      PUBLISH SNS alert TO authorities      STORE
severity ON incident record      RETURN severity
```

3.4 Dataset Details

Attribute	Description	Type
incident_id	Unique identifier for each incident report	String
citizen_id	Identifier of the reporting citizen (from Cognito)	String
latitude / longitude	GPS coordinates of the reported incident	Float

incident_type	Classifier output: flood / fire / structural damage	String
classifier_confidence	Confidence score returned by Rekognition	Float
severity_score	Computed weighted severity score	Float
status	unassigned / assigned / resolved	String
image_key	S3 object key of the uploaded photo	String
timestamp	Time of report submission	Datetime

Dataset source: this is a live application, not a batch-analytics pipeline, so the working dataset is whatever incident records get generated — partly from real citizen reports once the system is live, and partly from synthetic reports we generate through the Locust harness to simulate a disaster-scale surge for benchmarking, since we obviously do not have (and should not simulate) real citizen data at this stage. The image classification itself relies on Rekognition's pretrained model rather than any dataset we assembled ourselves.

3.5 Tools and Technologies

Layer	Technology
Frontend	React (citizen portal and admin portal), deployed via S3 and CloudFront
Authentication	Amazon Cognito (user pools)
API	Amazon API Gateway (REST endpoints)
Compute — Track A	AWS Lambda (report intake, image-classification trigger, query handler)
Compute — Track B	Single VM (Flask/Django monolith), hosted on Oracle Cloud Always Free
Database	Amazon DynamoDB (Track A) / self-managed PostgreSQL (Track B)
Image storage	Amazon S3 (pre-signed upload URLs, shared by both tracks)
AI / image classification	Amazon Rekognition (pretrained label detection)
Notifications	Amazon SNS (high-severity alerts)
Monitoring	Amazon CloudWatch (logs, metrics, alarms)
Security	IAM roles per service, least-privilege access
Load testing	Locust (simulated concurrent users, 10 to 500)
Programming language	Python 3.10+ (Lambda functions, Flask backend, Locust scripts), JavaScript/React (frontend)
IaC	AWS SAM (serverless deployment template)

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Experimental Setup

System configuration: Track A runs in a single AWS region. Track B runs on an Oracle Cloud Always Free virtual machine provisioned in that same geographic region, so that cross-region network latency does not quietly become a confounding variable. The Locust client itself runs on a separate host, so its own CPU and network usage do not eat into the resources of either deployment under test.

Tools used: AWS Lambda, API Gateway, DynamoDB, S3, Cognito, SNS and Rekognition for Track A; an Oracle Cloud virtual machine running Flask/Django with a self-managed PostgreSQL database for Track B; CloudWatch for monitoring both; and Locust, on Python 3.10+, for generating load and collecting the metrics.

4.2 Evaluation Methodology

Both tracks are hit with the identical Locust load profile, ramped through four stages — 10, 50, 200 and 500 simultaneous simulated users — each running the same sequence of requests: submit a report, upload an image, poll for status. At every stage, for every track, we record the p50/p95/p99 response latency, the error rate, and the cost per 1,000 requests, worked out from actual AWS billing for Track A and from the fixed hourly VM cost spread across request volume for Track B.

4.3 Performance Metrics

This project is a systems comparison rather than a predictive ML classifier, so the usual predictive metrics — accuracy, precision, F1 — only really apply to the Rekognition classification stage, not to the architecture comparison itself. For the architecture comparison, we use three metrics instead:

- Response latency (p50 / p95 / p99): how long a request takes, at the median and out at the tail, since the tail is usually where architectural differences under load show up first.
- Error rate: the share of requests that fail or time out at a given concurrency level — wherever this starts climbing sharply is roughly the practical capacity ceiling for that deployment.
- Cost per 1,000 requests: real AWS billing for Track A, versus the fixed VM cost amortised across request volume for Track B, so the two pricing models are compared directly instead of assumed.

4.4 Analysis Framework

Once Table 4.1 has real numbers in it, here is roughly how we plan to read them:

- If Track A's latency stays roughly flat as concurrency climbs while Track B's starts degrading past some point, that backs up the standard “serverless auto-scales, VM has fixed capacity” claim with an actual measurement instead of a repeated assertion.
- If Track A's cost curve climbs faster than Track B's at sustained high concurrency, that lines up with Jiang et al.'s (2021) finding that FaaS being faster does not mean it is cheaper.
- Any cold-start effect should show up as a spike in p95/p99 latency on Track A right at the start of each load stage, without a matching rise in p50 — which is the same pattern Sicari et al. (2023) reported.

We are writing this section in the conditional because, honestly, it has to stay that way until the benchmark in Table 4.1 has actually been run and we have real numbers to interpret rather than a plan for how we would interpret them.

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Conclusion

5.1.1 Summary of Work Done

This project designs and builds CloudRelief, a cloud-native disaster reporting and relief management platform where citizens can report an incident with location, description and a photo, and where the system estimates severity automatically through AI-based image classification before alerting authorities. What sets it apart from a normal single-architecture build is that the same citizen-reporting workflow is implemented twice — once as a fully serverless AWS deployment, once as a traditional always-on virtual machine — with a controlled Locust-based benchmarking setup built specifically to compare the two under identical simulated load.

5.1.2 Key Findings

Based on the evaluation framework set out in Chapter 4, this project is built to produce actual measured evidence — not an assumption borrowed from a slide deck — on how a serverless and a traditional deployment of the same disaster-reporting workload compare on latency, error rate and cost as concurrency scales from 10 to 500 users. The specific numbers belong in Table 4.1 once the benchmark has been run, read against the framework laid out in Section 4.4.

5.2 Future Work

5.2.1 Possible Enhancements

A few extensions seem worth doing later. A third track on Azure Functions would make this a genuinely multi-cloud comparison, which fits since the course syllabus covers both AWS and Azure. The Rekognition-based classifier could eventually be swapped for a custom model fine-tuned on region-specific disaster photos, closer to what Kyrkou and Theodorides (2019) did, once there is enough labelled data to make that worthwhile. The admin dashboard could also use a live heatmap of incident density, and the alerting logic could take cost and carbon intensity into account when choosing which region to route through.

5.2.2 Real-World Deployment Scope

Taking this beyond a course project would mean integrating with existing government emergency-dispatch systems and building in proper data-privacy safeguards for citizen location and image data — both outside what we could reasonably cover here. The benchmarking approach itself, though, is not tied to disaster reporting specifically; any team weighing serverless against a traditional deployment for a similarly bursty, safety-critical workload could reuse the same method.

REFERENCES

- Asghar, Tayyaba, et al. “Feasibility of Serverless Cloud Services for Disaster Management Information Systems.” 2018.
- Gupta, Ritwik, et al. “Creating xBD: A Dataset for Assessing Building Damage from Satellite Imagery.” Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, 2019.
- Jiang, Jiawei, Shaoduo Gan, Yue Liu, Fanlin Wang, Gustavo Alonso, Ana Klimovic, Ankit Singla, Wentao Wu, and Ce Zhang. “Towards Demystifying Serverless Machine Learning Training.” Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21), 2021, pp. 857–871.
- Kyrkou, Christos, and Theocharis Theocharides. “Deep-Learning-Based Aerial Image Classification for Emergency Response Applications Using Unmanned Aerial Vehicles.” CVPR Workshop on Computer Vision for UAVs, 2019.
- Nanda, et al. “Emergency Management Systems Using Mobile Cloud Computing: A Survey.” International Journal of Communication Systems, Wiley, 2023.
- Sicari, Christian, et al. “TEMA: Event Driven Serverless Workflows Platform for Natural Disaster Management.” 2023.
- Zhang, Song, Xiaochuan Luo, and Eugene Litvinov. “Serverless Computing for Cloud-Based Power Grid Emergency Generation Dispatch.” arXiv:2007.13733, 2020.



CloudRelief

A Smart Disaster Response & Relief Management System

Comparative Serverless vs. Traditional Cloud Architecture

CLOUD COMPUTING · BITE412L

Sanjay Chidambaram · Adithya Prabakaran · M R Yajunesh

23BIT0425

23BIT0102

23BIT0399



The Problem: When Every Second Counts

No Unified Reporting

During floods, fires, and structural emergencies, citizens have no fast, unified way to report incidents.

Scattered Intelligence

Rescue teams receive fragmented reports via calls and radio with no automatic way to distinguish urgent from non-urgent.

Manual Prioritisation

Response prioritisation is slow and manual — exactly when speed matters most.

Project Objectives

1

Cloud-Native Platform

Build a real-time disaster reporting and triage system designed for the cloud from the ground up.

2

AI Severity Classification

Automatically classify incident severity from citizen-submitted photos using AWS Rekognition.

3

Serverless Deployment

Deploy on AWS with a fully serverless, auto-scaling architecture – zero idle resources.

4

Empirical Comparison

Rigorously compare serverless vs. traditional deployment under real load conditions.

Proposed Solution: How CloudRelief Works

1

Report

Citizens submit incidents with description, GPS location, and photo via a React portal.

2

Classify

AI automatically identifies incident type and estimates severity from the uploaded image.

3

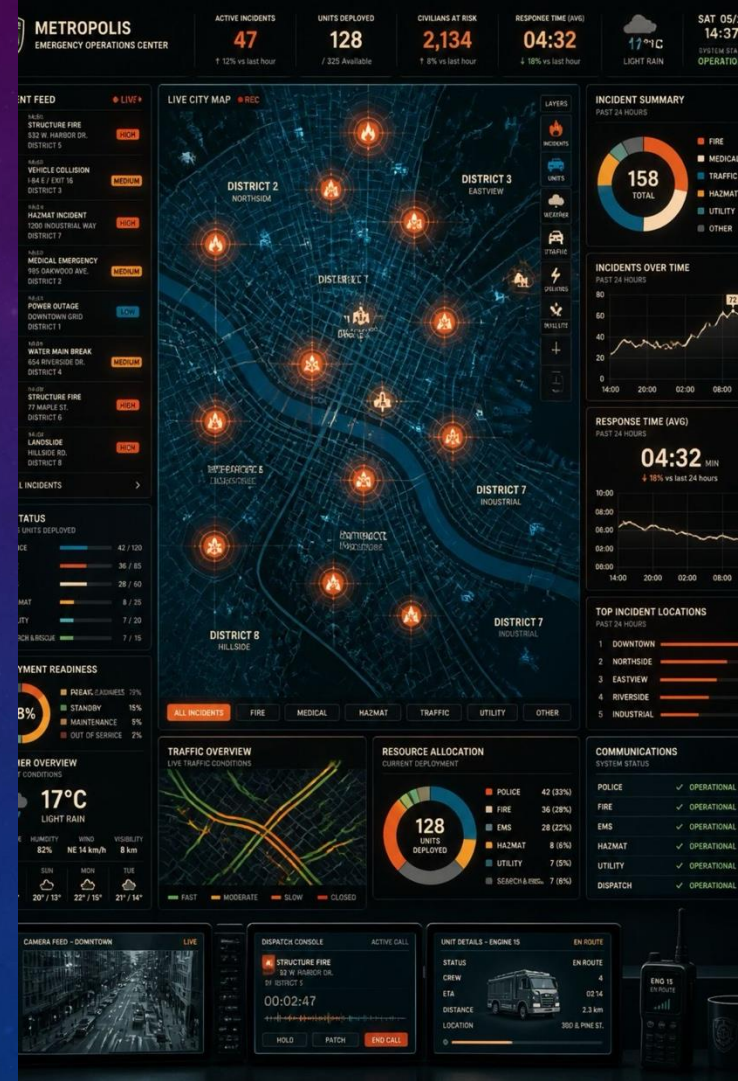
Alert

High-severity incidents trigger automatic alerts to the relevant authorities via SNS.

4

Respond

Admins use a live dashboard to view, prioritise, and assign rescue teams in real time.



Serverless Architecture



Citizen App Report

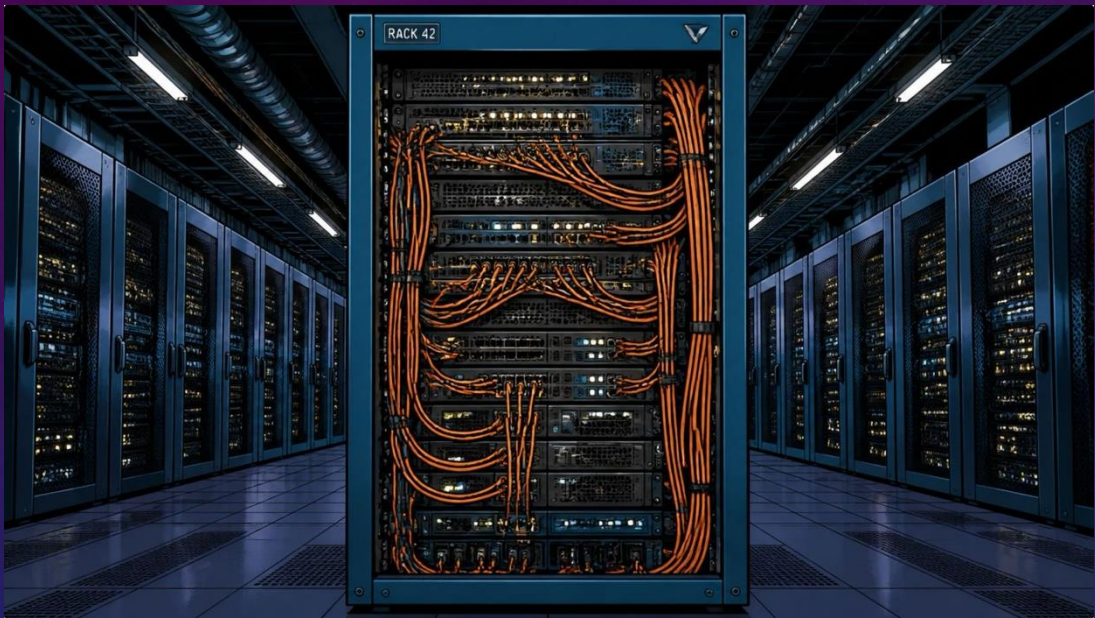
API Gateway Route

Lambda Processing

Store & Notify

Every component is event-driven and scales automatically. Lambda functions execute only when triggered — there is zero idle compute. CloudWatch provides unified observability across all services simultaneously.

Traditional Baseline Architecture



Track B mirrors the same citizen-facing report flow but runs on a fundamentally different compute model:

- ❑ S3 is shared across both tracks, ensuring storage is not a variable in the comparison.

Monolithic App

A single Flask/Django application deployed on one always-on VM.

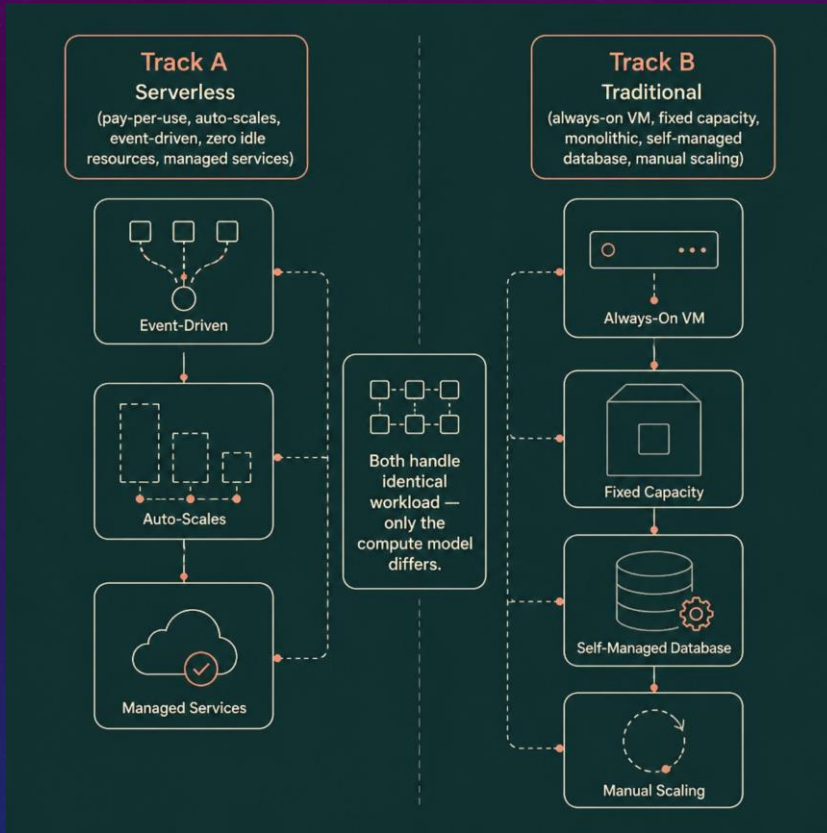
Fixed Capacity

No auto-scaling — capacity is static regardless of traffic spikes.

Self-Managed DB

Postgres database managed manually on the same VM.

Why Two Architectures?



This dual-track approach transforms CloudRelief from a build project into a **rigorous experiment**.

Both architectures handle the *identical* workload. The only variable is the compute model — making the comparison fair, meaningful, and empirically grounded.

Track A demonstrates
What "pay only when used, scales automatically" actually delivers in practice.

Track B represents
The traditional always-on approach that most legacy systems still rely on today.

AWS Services Used



Cognito & IAM

Secure identity management and fine-grained access control between services.



API Gateway & Lambda

Receives every request, routes it, and executes logic only when triggered – nothing idle.



DynamoDB & S3

Fully managed incident record storage and scalable image storage.



Rekognition & SNS

AI-powered image classification (flood, fire, structural damage) with automatic high-severity alerts.



CloudWatch

Unified logs and metrics across every service for full observability.

Key Features



Citizen Portal

Register, log in, report incidents with GPS and photo, and track response status in real time.



Admin Portal

View all incidents on a live map, assign rescue teams, and monitor severity statistics at a glance.



AI Component

Automatic image classification, severity scoring, and instant alerting for high-priority incidents.

Cloud Computing Concepts Demonstrated

CloudRelief is a practical demonstration of core cloud concepts, tested under real conditions across two contrasting architectures.

- Service Models
IaaS (Track B VM) vs. FaaS/Serverless (Track A Lambda)
- Virtualisation
VM-based deployment contrasted with function-level isolation
- Cloud Storage
Managed S3 & DynamoDB vs. self-hosted Postgres
- Security
IAM roles and Cognito authentication throughout
- Elasticity & Distributed Design
Auto-scaling under load and event-driven architecture





Benchmarking Methodology

Load Simulation

Locust load-testing harness simulates concurrent users: 10 → 50 → 200 → 500

Parallel Testing

Same requests fired at both Track A and Track B

Metrics Captured

Response time (p50/p95/p99), error rate, cost per 1,000 requests

Degradation Point

Identifies the load point where each architecture starts to degrade

Research Contribution

Working Title

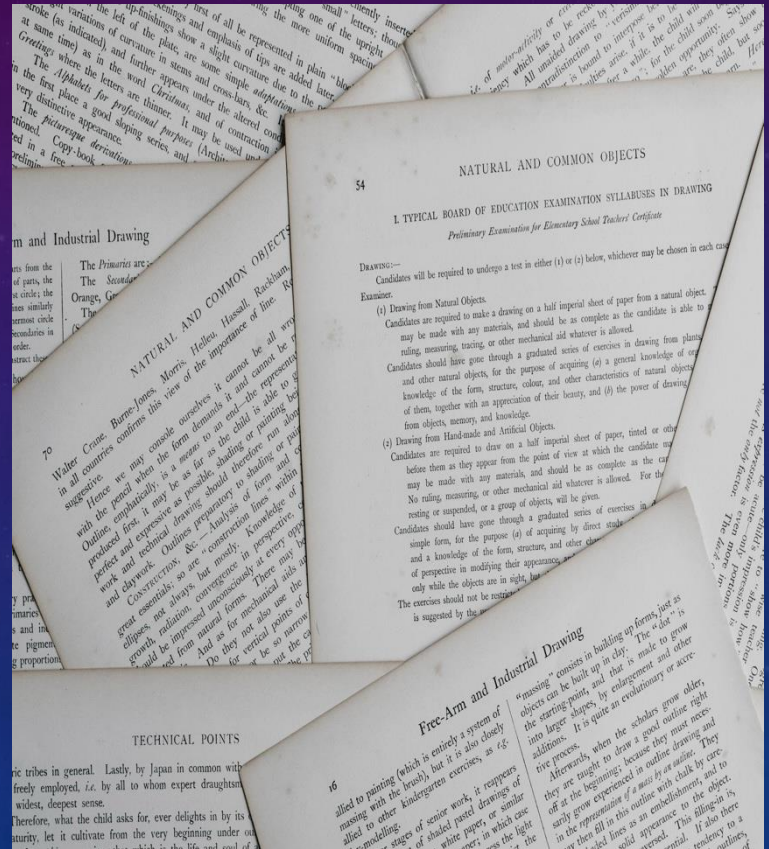
"Serverless vs. Traditional Cloud Architectures for Disaster Response Systems: A Comparative Performance and Cost Study"

Real Measured Data

Real measured data, not assumptions – both architectures built and tested identically

Publishable Empirical Study

Positions the project as a publishable empirical study, not just an application build



Feasibility & Cost

Free-Tier Services

Built entirely within free-tier limits (Lambda, DynamoDB, S3, SNS, CloudWatch, CloudFront)

Track B Hosting

Track B baseline hosted on Oracle Cloud Always Free to avoid EC2 billing uncertainty

Rekognition

Rekognition base label detection used instead of Custom Labels to avoid training costs

Domain

Custom domain (Route53) skipped – default AWS URLs used

Team & Roles

Sanjay Chidambaram

Serverless Backend & AI/ML

Adithya Prabakaran

Frontend & Authentication

M R Yajunesh

Infrastructure, Benchmarking & Research

Conclusion

→ Working Platform

A working disaster response platform, deployed on real cloud infrastructure

→ Empirical Comparison

A genuine empirical comparison backing a research paper, not just a demo

→ Syllabus Coverage

Directly demonstrates 6 of 8 course syllabus modules in one system